Practical Session 4 : NLP Pipeline - Part 2

Tokenisation → Stopwords → Normalisation → Vectorisation → L2 Normalisation

Dr. BENDIABDALLAH M.H
Université Belhadj BOUCHAIB - Ain Temouchent

Module: Natural Language Processing (NLP)

Learning Outcomes

By the end of this practical session, you will be able to:

- Load a long text from a **separate .txt file**.
- Apply basic and rule-based tokenisation.
- Remove stopwords and justify the choice (including negations).
- Normalise words using stemming or lemmatisation.
- Vectorise text using `CountVectorizer` (unigrams and (1,2)-grams).
- Apply **L2 normalisation** to compare text segments using cosine similarity.

1 Step 4 — Vectorisation with CountVectorizer (unigrams then (1,2)-grams)

We will implement a **custom analyser** to control preprocessing.

4A. Custom analyser

```
def custom_analyser(text: str):
    """Analyzer for unigrams only"""
    # 1) tokenise
    tokens = rule_based_tokenise(text)

    # 2) keep alphabetic tokens + lowercase
    tokens = [t.lower() for t in tokens if t.isalpha()]

    # 3) stopwords removal
    tokens = remove_stopwords(tokens, stop_words)

    # 4) normalisation: choose ONE (uncomment ONE)
    # tokens = stem_tokens(tokens)
    tokens = lemmatise_tokens(tokens) # <-- if you choose lemmatisation

    return tokens


def custom_analyser_with_bigrams(text: str):
```

Dr. BENDIABDALLAH M.H

```
    """Analyzer that generates both unigrams and bigrams"""
    # Get unigrams
    tokens = custom_analyser(text)

    # Generate bigrams
    bigrams = [f"{tokens[i]} {tokens[i+1]}" for i in range(len(tokens)-1)]

    # Return both unigrams and bigrams
    return tokens + bigrams


print("Unigrams (first 30):")
print(custom_analyser(docs[0])[:30])
print("\nWith bigrams (first 40):")
print(custom_analyser_with_bigrams(docs[0])[:40])
```

Question 4 (answer in code):

- Compute and print the number of unique tokens in Document 1 **before** preprocessing and **after** preprocessing. What do you observe?

```
# ANSWER Q4: (write code + short comment)
```

4B. CountVectorizer with unigrams

```
from sklearn.feature_extraction.text import CountVectorizer
import pandas as pd

cv_uni = CountVectorizer(analyzer=custom_analyser, ngram_range=(1,1))
X_uni = cv_uni.fit_transform(docs)

terms_uni = cv_uni.get_feature_names_out()
df_uni = pd.DataFrame(X_uni.toarray(), columns=terms_uni)

print("Unigram matrix shape:", X_uni.shape)
df_uni.head()
```

4C. CountVectorizer with (1,2)-grams

```
# Use the custom analyzer that generates both unigrams and bigrams
cv_12 = CountVectorizer(analyzer=custom_analyser_with_bigrams)
X_12 = cv_12.fit_transform(docs)

terms_12 = cv_12.get_feature_names_out()
df_12 = pd.DataFrame(X_12.toarray(), columns=terms_12)

print("1-2 gram matrix shape:", X_12.shape)
print("\nExample unigrams (first 20):", [t for t in terms_12 if ' ' not in t][:20])

print("\nExample bigrams (first 20):", [t for t in terms_12 if ' ' in t][:20])
df_12.head()
```

Question (answer in code):

- Find and print **3 meaningful bigrams** present in the features. Explain briefly why bigrams can capture more context than unigrams.

```
# ANSWER : ...
```

Dr. BENDIABDALLAH M.H

2 Step 5 — L2 Normalisation of Vectors + Cosine Similarity

5A. Apply L2 normalisation

```
from sklearn.preprocessing import normalize
import numpy as np

X_uni_l2 = normalize(X_uni, norm="l2")
X_12_l2 = normalize(X_12, norm="l2")

# Verify norms
norms = np.sqrt((X_uni_l2.multiply(X_uni_l2)).sum(axis=1))
print("L2 norms (unigram):", np.round(np.asarray(norms).ravel(), 4))
```

5B. Compare documents with cosine similarity

```
from sklearn.metrics.pairwise import cosine_similarity

sim_uni = cosine_similarity(X_uni_l2)
sim_12 = cosine_similarity(X_12_l2)

print("Cosine similarity (unigrams):\n", np.round(sim_uni, 3))
print("\nCosine similarity (1-2 grams):\n", np.round(sim_12, 3))
```

Question (answer in code):

- Which pair of documents (paragraphs) is most similar with unigrams? Does the most similar pair change with (1,2)-grams? Explain in 2–3 lines.

```
# ANSWER Q6: ...
```

Dr.BENDIABDALLAH M.H